Introduction to NoSQL databases

This introduction walks through the CRUD (create, read, update, delete) operations on `mongodb`.

MongoDB

MongoDB is a document-oriented NoSQL database. It stores data in JSON-like documents with dynamic schemas, making the integration of data in certain types of applications easier and faster.

We will be connecting to a MongoDB cluster hosted on MongoDB Atlas. MongoDB Atlas is a cloud database service that allows you to host MongoDB databases on the cloud.

Prerequisite: The learner is required to set up an account on MongoDB here and set up a (free tier) cluster. Please take note of the cluster URL and credentials (DB username and DB password) required to access the cluster. If necessary, you can refer to the screenshots below:

step 0 (Choose **free** cluster and 'Create Deployment')

step 1 (When prompted, set up your DB username and DB password - note it down! Note this is different from your login username and password.)

step 2 (Click 'Drivers' to see how to access using python)

step 3 (Copy and paste the code into a cell below - note you have to input your password as well)

We will be using the `pymongo` library to connect to the MongoDB database.

Note that to run the command `python -m pip install "pymongo[srv]"` in a Jupyter notebook cell, you have to add an exclamation mark as such: `!python -m pip install "pymongo[srv]"`

```
# Copy and paste the pip install step from step 3 above into this cell below
# ------------------------------------------------------------------------
!python -m pip install "pymongo[srv]"
```

```
Requirement already satisfied: pymongo[srv] in /home/annie/miniconda3/envs/bd
WARNING: pymongo 4.15.4 does not provide the extra 'srv'
Requirement already satisfied: dnspython<3.0.0,>=1.16.0 in /home/annie/minico
```

```
Start coding or generate with AI.
```

```
Pinged your deployment. You successfully connected to MongoDB!
```

Connecting to MongoDB

Connection Code

```
# Copy and paste the connection code from step 3 above into this cell below
# ------------------------------------------------------------------------

# sometimes the uri below did not include password, it has a place holder <p
# make sure you complete uri with password
# delete the following when .env file are setup.
```

IMPORTANT

Your connection secrets including the password is yours. Do not share the secrets above. Do not sync the information above to your public repository. To safely keep your password and connection string locally. Please run the following cell.

> Please refer to the end of MongoDB installation guide for troubleshooting and resetting password.

Saving Connection Secrets Locally Using dotenv (Run Once)

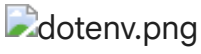
Run the following cell to save your secrets to a local dotenv file:

- Please ensure that your connection is successful in the previous cell
- You only need to run once

```
# Cell: Export MongoDB configuration to .env file
import os

# Check if uri exists and is not null
try:
    # 1. Check if 'uri' is defined in the current scope (`locals()`)
    # 2. Check if the value of 'uri' is not empty (e.g., not None, not an em
    if 'uri' not in locals() or not uri:
        # If either condition fails, raise a custom exception immediately.
        # This will be caught by the outer 'except' block.
        raise ValueError("MongoDB uri is not defined or is empty. Please run

    # Create .env file with MongoDB configuration
    env_content = f"""# MongoDB Configuration
# Auto-generated from notebook
# This file will not be synced to Github
# To save reconfiguration effort, please make a copy to somewhere safe
MONGODB_URI='{uri}'
"""

    with open('../.env', 'w') as f:
        f.write(env_content)

    print("✅ Configuration exported to .env file")

except ValueError as e:
    print(f"❌ Error: {e}")
```

```
✅ Configuration exported to .env file
```

You should see a file called `.env`, similar to the screenshot below:



Testing MongoDB Connection Using dotenv

You can start from here if you have already setup the env file.

```
import pymongo
import os
from dotenv import load_dotenv
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi

# Load environment variables from .env file
load_dotenv()
MONGODB_URI = os.getenv('MONGODB_URI')
client = MongoClient(MONGODB_URI, server_api=ServerApi('1'))
# Send a ping to confirm a successful connection
try:
```

```
        client.admin.command('ping')
        print("Pinged your deployment. You successfully connected to MongoDB!")
    except Exception as e:
        print(e)
```

```
Pinged your deployment. You successfully connected to MongoDB!
```

IMPORTANT

If your connection is good using dotenv. Please remove what you have pasted previously on the `Connection Code`.

If you have connected to the cluster successfully above, you can skip the below cell. Else, set up the connection to the cluster (replace , , below with your own)

```
# client = pymongo.MongoClient("mongodb+srv://<DB-USERNAME>:<DB-PASSWORD>@<C
```

Accessing Sample Database

A cluster can host multiple databases. List all databases in the cluster:

```
client.list_database_names()
```

```
['sample_mflix', 'admin', 'local']
```

Read

You can access a database using attribute style access:

```
db = client.sample_mflix
```

A collection is a group of documents stored in MongoDB, and can be thought of as roughly the equivalent of a table in a relational database.

List all collections in the database:

```
db.list_collection_names()
```

```
['comments', 'users', 'embedded_movies', 'theaters', 'sessions', 'movies']
```

Let's assign the `movies` collection to a variable:

```
movies = db.movies
```

Data in MongoDB is represented (and stored) using JSON-style documents. In PyMongo we use dictionaries to represent documents.

The most basic type of query that can be performed in MongoDB is `find_one()`. This method returns a single document matching a query (or None if there are no matches). You can also omit the query to get the first document from the collection:

```
movies.find_one()
```

```
{'_id': ObjectId('573a1390f29313caabcd42e8'),
 'plot': 'A group of bandits stage a brazen train hold-up, only to find a
determined posse hot on their heels.',
 'genres': ['Short', 'Western'],
 'runtime': 11,
 'cast': ['A.C. Abadie',
  "Gilbert M. 'Broncho Billy' Anderson",
  'George Barnes',
  'Justus D. Barnes'],
 'poster': 'https://m.media-
amazon.com/images/M/MV5BMTU3NjE5NzYtYTYyNS00MDVmLWIwYjgtMmYwYWIxZDYyNzU2XkEyX
 'title': 'The Great Train Robbery',
 'fullplot': "Among the earliest existing films in American cinema - notable
as the first film that presented a narrative story to tell - it depicts a
group of cowboy outlaws who hold up a train and rob the passengers. They are
then pursued by a Sheriff's posse. Several scenes have color included - all
hand tinted.",
 'languages': ['English'],
 'released': datetime.datetime(1903, 12, 1, 0, 0),
 'directors': ['Edwin S. Porter'],
 'rated': 'TV-G',
 'awards': {'wins': 1, 'nominations': 0, 'text': '1 win.'},
 'lastupdated': '2015-08-13 00:27:59.177000000',
 'year': 1903,
 'imdb': {'rating': 7.4, 'votes': 9847, 'id': 439},
 'countries': ['USA'],
 'type': 'movie',
 'tomatoes': {'viewer': {'rating': 3.7, 'numReviews': 2559, 'meter': 75},
  'fresh': 6,
  'critic': {'rating': 7.6, 'numReviews': 6, 'meter': 100},
  'rotten': 0,
  'lastUpdated': datetime.datetime(2015, 8, 8, 19, 16, 10)},
 'num_mflix_comments': 0}
```

> Get the first document from the `comments` collection.

You can pass a query to the `find_one()` method to filter the results to only include documents that match the query criteria. The first argument to the find() method is a document containing the query criteria. To specify an equality match, use a document (dictionary) with the specific field and value:

```
movies.find_one({'title': 'Traffic in Souls'})
```

```
{'_id': ObjectId('573a1390f29313caabcd4eaf'),
 'plot': 'A woman, with the aid of her police officer sweetheart, endeavors
to uncover the prostitution ring that has kidnapped her sister, and the
philanthropist who secretly runs it.',
 'genres': ['Crime', 'Drama'],
 'runtime': 88,
 'cast': ['Jane Gail', 'Ethel Grandin', 'William H. Turner', 'Matt Moore'],
 'num_mflix_comments': 1,
 'poster': 'https://m.media-
amazon.com/images/M/MV5BYzk0YWQzMGYtYTM5MC00NjM2LWE5YzYtMjgyNDVhZDg1N2YzXkEyX
 'title': 'Traffic in Souls',
 'lastupdated': '2015-09-15 02:07:14.247000000',
 'languages': ['English'],
 'released': datetime.datetime(1913, 11, 24, 0, 0),
 'directors': ['George Loane Tucker'],
 'rated': 'TV-PG',
 'awards': {'wins': 1, 'nominations': 0, 'text': '1 win.'},
 'year': 1913,
 'imdb': {'rating': 6.0, 'votes': 371, 'id': 3471},
 'countries': ['USA'],
 'type': 'movie',
 'tomatoes': {'viewer': {'rating': 3.0, 'numReviews': 85, 'meter': 57},
  'dvd': datetime.datetime(2008, 8, 26, 0, 0),
  'lastUpdated': datetime.datetime(2015, 8, 10, 18, 33, 55)}}
```

You can also query by `_id` which is the unique identifier (think of it as a primary key) for each document in a collection. However, it is an `ObjectId` hence you need to convert it from a string:

```
from bson.objectid import ObjectId
```

```
movies.find_one({'_id': ObjectId('573a1390f29313caabcd4eaf')})
```

```
{'_id': ObjectId('573a1390f29313caabcd4eaf'),
 'plot': 'A woman, with the aid of her police officer sweetheart, endeavors
to uncover the prostitution ring that has kidnapped her sister, and the
philanthropist who secretly runs it.',
 'genres': ['Crime', 'Drama'],
 'runtime': 88,
 'cast': ['Jane Gail', 'Ethel Grandin', 'William H. Turner', 'Matt Moore'],
 'num_mflix_comments': 1,
 'poster': 'https://m.media-
amazon.com/images/M/MV5BYzk0YWQzMGYtYTM5MC00NjM2LWE5YzYtMjgyNDVhZDg1N2YzXkEyX
 'title': 'Traffic in Souls',
 'lastupdated': '2015-09-15 02:07:14.247000000',
 'languages': ['English'],
 'released': datetime.datetime(1913, 11, 24, 0, 0),
 'directors': ['George Loane Tucker'],
 'rated': 'TV-PG',
 'awards': {'wins': 1, 'nominations': 0, 'text': '1 win.'},
 'year': 1913,
 'imdb': {'rating': 6.0, 'votes': 371, 'id': 3471},
```

```
 'countries': ['USA'],
 'type': 'movie',
 'tomatoes': {'viewer': {'rating': 3.0, 'numReviews': 85, 'meter': 57},
  'dvd': datetime.datetime(2008, 8, 26, 0, 0),
  'lastUpdated': datetime.datetime(2015, 8, 10, 18, 33, 55)}}
```

> Get the document with the following `plot`: `"A romantic rivalry among members of a secret society becomes even tenser when one of the men is assigned to carry out an assassination."`

To get more than a single document as the result of a query we use the `find()` method. `find()` returns a Cursor instance, which allows us to iterate over all matching documents.

We can limit the number of results returned using `limit()`:

```
for m in movies.find().limit(5):
    print(m)
```

```
{'_id': ObjectId('573a1390f29313caabcd42e8'), 'plot': 'A group of bandits sta
{'_id': ObjectId('573a1390f29313caabcd446f'), 'plot': "A greedy tycoon decide
{'_id': ObjectId('573a1390f29313caabcd4803'), 'plot': 'Cartoon figures announ
{'_id': ObjectId('573a1390f29313caabcd4eaf'), 'plot': 'A woman, with the aid
{'_id': ObjectId('573a1390f29313caabcd50e5'), 'plot': 'The cartoonist, Winsor
```

We can use `query operators` to perform more complex queries. For example, we can use the `$gt` operator to find documents where the `released` date is greater (later) than `2015-12-01`.

For more information on query operators, refer to the MongoDB documentation.

```
from datetime import datetime

d = datetime(2015, 12, 1)

for m in movies.find({"released": {"$gt": d}}):
    print(m)
```

```
{'_id': ObjectId('573a13cef29313caabd88223'), 'plot': 'Sun Wukong, (The Monke
{'_id': ObjectId('573a13d6f29313caabda10e6'), 'fullplot': 'Rick is a slave to
{'_id': ObjectId('573a13d8f29313caabda63c2'), 'plot': 'The Dark Horse is an e
{'_id': ObjectId('573a13d9f29313caabda885a'), 'plot': 'In rural Indiana, Noah
{'_id': ObjectId('573a13d9f29313caabdaa555'), 'plot': "1892, Saint Petersburg
{'_id': ObjectId('573a13daf29313caabdad539'), 'plot': 'An unexpected romance
{'_id': ObjectId('573a13e9f29313caabdcc4ae'), 'plot': 'Some of the chapters f
{'_id': ObjectId('573a13e9f29313caabdcc6c6'), 'plot': 'A futuristic love stor
{'_id': ObjectId('573a13e9f29313caabdcd0d3'), 'plot': 'Fred and Mick, two old
{'_id': ObjectId('573a13edf29313caabdd41ff'), 'plot': 'Celina works at a remo
{'_id': ObjectId('573a13f0f29313caabdd969c'), 'plot': 'The darkest chapter of
```

```
{'_id': ObjectId('573a13f0f29313caabdda7ac'), 'plot': 'China, 1999. Childhood
{'_id': ObjectId('573a13f0f29313caabddab02'), 'plot': 'Filmmakers discuss how
{'_id': ObjectId('573a13f1f29313caabddc788'), 'plot': 'In the horror of 1944
{'_id': ObjectId('573a13f2f29313caabddd3b6'), 'plot': 'In the well-to-do subu
{'_id': ObjectId('573a13f4f29313caabde0bfd'), 'plot': "Anna suffers from agor
{'_id': ObjectId('573a13f7f29313caabde74c6'), 'plot': 'An aristocratic brothe
{'_id': ObjectId('573a13f7f29313caabde74df'), 'plot': "A hot summer's day in
{'_id': ObjectId('573a13f8f29313caabde7d77'), 'countries': ['France'], 'genre
{'_id': ObjectId('573a13f8f29313caabde8d7a'), 'plot': 'Costi leads a peaceful
{'_id': ObjectId('573a13f9f29313caabdeb527'), 'plot': 'After having left a lo
```

> Return the documents with `released` date between `2015-12-01` and `2015-12-15`.

We can do a regex search using the `$regex` operator. Let's search for all the movies with `"spy"` in the plot.

```
for m in movies.find({"plot": {"$regex": "spy"}}):
    print(m)
```

```
{'_id': ObjectId('573a1391f29313caabcd962d'), 'plot': 'The Austrian Secret S
{'_id': ObjectId('573a1392f29313caabcd9ed7'), 'plot': 'Chester Kent struggle
{'_id': ObjectId('573a1393f29313caabcdc0cb'), 'plot': 'Aboard the freighter
{'_id': ObjectId('573a1393f29313caabcdcb64'), 'plot': "During the Nazi occup
{'_id': ObjectId('573a1393f29313caabcdd482'), 'plot': 'Bill Dietrich becomes
{'_id': ObjectId('573a1393f29313caabcdd679'), 'plot': 'A young Irish woman h
{'_id': ObjectId('573a1393f29313caabcdd86b'), 'plot': 'A woman is asked to s
{'_id': ObjectId('573a1394f29313caabcdf069'), 'plot': 'The Jeffersons are th
{'_id': ObjectId('573a1394f29313caabcdf539'), 'plot': 'A pickpocket unwittin
{'_id': ObjectId('573a1394f29313caabce0138'), 'plot': 'During the Civil War
{'_id': ObjectId('573a1395f29313caabce2c28'), 'plot': 'After a series of mis
{'_id': ObjectId('573a1395f29313caabce2cfe'), 'plot': 'A spy spoof in the 60
{'_id': ObjectId('573a1395f29313caabce2fc4'), 'plot': 'In an early spy spoof
{'_id': ObjectId('573a1396f29313caabce3d78'), 'plot': 'A French intelligence
{'_id': ObjectId('573a1396f29313caabce4fcb'), 'plot': 'A martial artist agre
{'_id': ObjectId('573a1396f29313caabce54c3'), 'fullplot': 'Harry Caul is a c
{'_id': ObjectId('573a1397f29313caabce754f'), 'plot': "A reflection of Russi
{'_id': ObjectId('573a1397f29313caabce7e36'), 'plot': 'A ruthless German spy
{'_id': ObjectId('573a1397f29313caabce81cb'), 'plot': 'A visiting dignitary,
{'_id': ObjectId('573a1397f29313caabce8896'), 'plot': 'A dramatization of th
{'_id': ObjectId('573a1397f29313caabce8a2f'), 'plot': "Actress Coral Browne
{'_id': ObjectId('573a1398f29313caabce8fdc'), 'plot': "A young actor's obses
{'_id': ObjectId('573a1398f29313caabce9771'), 'plot': "After a newspaper rep
{'_id': ObjectId('573a1398f29313caabceb109'), 'plot': '1908: Pascali, a spy
{'_id': ObjectId('573a1398f29313caabceb681'), 'plot': 'Another planet in the
{'_id': ObjectId('573a1399f29313caabcec1b0'), 'plot': 'Convicted felon Nikit
{'_id': ObjectId('573a139af29313caabcef480'), 'plot': 'A retired British spy
{'_id': ObjectId('573a139af29313caabcef76b'), 'plot': "Sharpe is tasked to p
{'_id': ObjectId('573a139af29313caabcefe3a'), 'plot': 'A martial artist agre
{'_id': ObjectId('573a139af29313caabceff3f'), 'plot': "Harriet M. Welsch is
{'_id': ObjectId('573a139af29313caabcf0178'), 'fullplot': 'Based on the hit
{'_id': ObjectId('573a139bf29313caabcf371c'), 'plot': 'A visiting dignitary,
{'_id': ObjectId('573a139df29313caabcfa6ce'), 'fullplot': "Five years after
{'_id': ObjectId('573a139ff29313caabd01d0b'), 'fullplot': "North Korea's 8th
```

```
{'_id': ObjectId('573a13a2f29313caabd0a214'), 'plot': "The landlord of a boa
{'_id': ObjectId('573a13a2f29313caabd0c2c0'), 'plot': 'A tailor living in Pa
{'_id': ObjectId('573a13a5f29313caabd15003'), 'plot': "After a sudden attacl
{'_id': ObjectId('573a13a6f29313caabd17899'), 'plot': 'An unsuspecting, dise
{'_id': ObjectId('573a13a6f29313caabd182f2'), 'plot': 'The Cortez siblings s
{'_id': ObjectId('573a13abf29313caabd25f52'), 'plot': 'In 1934, four brillia
{'_id': ObjectId('573a13abf29313caabd262e6'), 'plot': 'Arun Khanna is a spy
{'_id': ObjectId('573a13aef29313caabd2c8f8'), 'plot': 'France, 1936-37. The
{'_id': ObjectId('573a13b2f29313caabd3839c'), 'fullplot': "American Maxwell
{'_id': ObjectId('573a13b5f29313caabd42080'), 'plot': "Hallam's talent for s
{'_id': ObjectId('573a13bcf29313caabd558be'), 'fullplot': 'Evelyn Salt is a
{'_id': ObjectId('573a13bdf29313caabd59275'), 'fullplot': "June Havens finds
{'_id': ObjectId('573a13bff29313caabd5dfdd'), 'fullplot': "Based on Martin M
{'_id': ObjectId('573a13bff29313caabd6075f'), 'fullplot': "Based on Martin M
{'_id': ObjectId('573a13c2f29313caabd68276'), 'fullplot': "Former CIA spy Bo
{'_id': ObjectId('573a13c3f29313caabd69004'), 'fullplot': 'In the age-old ba
{'_id': ObjectId('573a13c5f29313caabd6fe15'), 'plot': 'A factory worker, Dou
{'_id': ObjectId('573a13c6f29313caabd71b22'), 'fullplot': 'The high stakes t
{'_id': ObjectId('573a13c6f29313caabd73594'), 'plot': '1942, Nanjing (Nankir
{'_id': ObjectId('573a13c9f29313caabd790d3'), 'plot': 'A retired spy is call
{'_id': ObjectId('573a13c9f29313caabd7b7f1'), 'plot': 'Young spy Harriet Wel
{'_id': ObjectId('573a13d9f29313caabda9158'), 'plot': 'A young woman finds c
{'_id': ObjectId('573a13daf29313caabdacb61'), 'plot': 'The son of a foundin
```

> Return the documents with the `plot` that starts with `"Once upon a time"`.

You can sort by any field in the document. The default is ascending order, but you can specify descending order by using the `pymongo.DESCENDING` constant.

```
for m in movies.find({"plot": {"$regex": "spy"}}).sort('released', pymongo.D
    print(f"{m['title']} was released in {m['released']}")
```

```
Jack Strong was released in 2015-07-24 00:00:00
Restless was released in 2015-05-15 00:00:00
Kingsman: The Secret Service was released in 2015-02-13 00:00:00
Rosewater was released in 2014-11-27 00:00:00
The Green Prince was released in 2014-11-27 00:00:00
Open Windows was released in 2014-10-02 00:00:00
Paranoia was released in 2013-08-16 00:00:00
Total Recall was released in 2012-08-03 00:00:00
The Spy was released in 2012-04-05 00:00:00
Spy Kids: All the Time in the World in 4D was released in 2011-08-19 00:00:00
```

> Return the documents with the `plot` that starts with `"Once upon a time"` in ascending order of released date, print only title, plot and released fields.

MongoDB Aggregation

MongoDB's `aggregation` pipelines are one of its most powerful features. They allow you to write expressions, broken down into a series of stages, which perform operations including aggregation, transformations, and joins. This allows you to do calculations and analytics across documents and collections.

```
pipeline = [
   {
      "$match": {
         "title": "A Star Is Born"
      }
   },
   {
      "$sort": {
         "year": pymongo.ASCENDING
      }
   },
]
results = movies.aggregate(pipeline)

for movie in results:
   print(" * {title}, {first_castmember}, {year}".format(
         title=movie["title"],
         first_castmember=movie["cast"][0],
         year=movie["year"],
   ))
```

```
 * A Star Is Born, Judy Garland, 1954
 * A Star Is Born, Barbra Streisand, 1976
```

This pipeline above has two stages.

- The first is a `$match` stage, which is similar to querying a collection with find(). It filters the documents passing through the stage based on the query. Because it's the first stage in the pipeline, its input is all of the documents in the movie collection.
- The second stage is a `$sort` stage. Only the documents for the movie "A Star Is Born" are passed to this stage, so the result will be all of the movies called "A Star Is Born," now sorted by their year field, with the oldest movie first.

Finally, calls to `aggregate()` return a cursor pointing to the resulting documents.

You can also use `$lookup` with `aggregate` to query movies and embed the related comments, like a JOIN in a relational database:

```
# Look up related documents in the 'comments' collection:
stage_lookup_comments = {
   "$lookup": {
         "from": "comments",
         "localField": "_id",
```

```
            "foreignField": "movie_id",
            "as": "related_comments",
      }
   }

   # Limit to the first 5 documents:
   stage_limit_5 = { "$limit": 5 }

   pipeline = [
      stage_lookup_comments,
      stage_limit_5,
   ]

   results = movies.aggregate(pipeline)
   for movie in results:
      print(movie['title'])
      for comment in movie["related_comments"][:5]:
            print(" * {name}: {text}".format(
               name=comment["name"],
               text=comment["text"]))
      print()
```

```
The Great Train Robbery

A Corner in Wheat
 * John Bishop: Id error ab at molestias dolorum incidunt. Non deserunt praes
Minus dicta numquam quasi. Rem totam cumque at eum. Ullam hic ut ea magni.

Winsor McCay, the Famous Cartoonist of the N.Y. Herald and His Moving Comics

Traffic in Souls
 * Taylor Scott: Iure laboriosam quo et necessitatibus sed. Id iure delectus

Gertie the Dinosaur
```

The lookup above functions like a left join, some of the movies do not have any comments.

To do something similar to an inner join, add some stages to match only movies which have at least one comment.

```
   # Calculate the number of comments for each movie:
   stage_add_comment_count = {
      "$addFields": {
            "comment_count": {
               "$size": "$related_comments"
            }
      }
   }

   # Match movie documents with at least 1 comment:
   stage_match_with_comments = {
```

```
    "$match": {
          "comment_count": {
             "$gte": 1
          }
    }
}
```

```
pipeline = [
   stage_lookup_comments,
   stage_add_comment_count,
   stage_match_with_comments,
   stage_limit_5,
]

results = movies.aggregate(pipeline)
for movie in results:
   print(movie["title"])
   print("Comment count:", movie["comment_count"])

   for comment in movie["related_comments"][:5]:
         print(" * {name}: {text}".format(
            name=comment["name"],
            text=comment["text"]))
   print()
```

```
The Ace of Hearts
Comment count: 1
 * John Bishop: Accusamus qui distinctio ut ab saepe tenetur. Quae optio aut

The Four Horsemen of the Apocalypse
Comment count: 1
 * Olenna Tyrell: Sed iste tenetur ut. Veritatis deserunt iusto blanditiis si

A Corner in Wheat
Comment count: 1
 * John Bishop: Id error ab at molestias dolorum incidunt. Non deserunt praes
Minus dicta numquam quasi. Rem totam cumque at eum. Ullam hic ut ea magni.

Traffic in Souls (1913)
Comment count: 1
 * Taylor Scott: Iure laboriosam quo et necessitatibus sed. Id iure delectus

High and Dizzy
Comment count: 1
 * Yolanda Owen: Occaecati commodi quidem aliquid delectus dolores. Facilis f
```

> Repeat the above but with movies that have more than 2 comments.

Finally, you can do "group by" operations too. Let's group by the `year` and count the number of movies in each year:

```
stage_group_year = {
   "$group": {
         "_id": "$year",
         # Count the number of movies in the group:
         "movie_count": { "$sum": 1 },
   }
}

pipeline = [
   stage_group_year,
]
results = movies.aggregate(pipeline)

# Loop through the 'year-summary' documents:
for year_summary in results:
   print(year_summary)
```

```
{'_id': 1995, 'movie_count': 372}
{'_id': '1995è', 'movie_count': 1}
{'_id': '2007è', 'movie_count': 3}
{'_id': 1973, 'movie_count': 112}
{'_id': 1928, 'movie_count': 8}
{'_id': 2013, 'movie_count': 1105}
{'_id': '1987è', 'movie_count': 1}
{'_id': 2016, 'movie_count': 1}
{'_id': 1960, 'movie_count': 73}
{'_id': 1983, 'movie_count': 161}
{'_id': 1942, 'movie_count': 30}
{'_id': 2012, 'movie_count': 955}
{'_id': '2009è', 'movie_count': 2}
{'_id': 1934, 'movie_count': 23}
{'_id': '1999è', 'movie_count': 1}
{'_id': 1896, 'movie_count': 2}
{'_id': '2012è', 'movie_count': 3}
{'_id': 1955, 'movie_count': 67}
{'_id': '2006è', 'movie_count': 1}
{'_id': 1951, 'movie_count': 54}
{'_id': 1998, 'movie_count': 513}
{'_id': 2009, 'movie_count': 917}
{'_id': 2003, 'movie_count': 603}
{'_id': 2000, 'movie_count': 581}
{'_id': 1975, 'movie_count': 107}
{'_id': 1977, 'movie_count': 123}
{'_id': 1920, 'movie_count': 4}
{'_id': 1952, 'movie_count': 45}
{'_id': 1996, 'movie_count': 407}
{'_id': 1916, 'movie_count': 2}
{'_id': 1923, 'movie_count': 2}
{'_id': 2002, 'movie_count': 622}
{'_id': 1941, 'movie_count': 24}
{'_id': 1978, 'movie_count': 128}
{'_id': 1997, 'movie_count': 439}
{'_id': 2004, 'movie_count': 678}
{'_id': 1925, 'movie_count': 3}
{'_id': 1931, 'movie_count': 20}
{'_id': '1997è', 'movie_count': 2}
{'_id': 1924, 'movie_count': 6}
{'_id': 1929, 'movie_count': 7}
```

```
{'_id': 1966, 'movie_count': 87}
{'_id': 2007, 'movie_count': 810}
{'_id': 1954, 'movie_count': 47}
{'_id': 1974, 'movie_count': 103}
{'_id': 1917, 'movie_count': 2}
{'_id': 1963, 'movie_count': 69}
{'_id': '1988è', 'movie_count': 1}
{'_id': 1972, 'movie_count': 121}
{'_id': 1953, 'movie_count': 65}
{'_id': 1930, 'movie_count': 10}
{'_id': 1987, 'movie_count': 222}
{'_id': 1940, 'movie_count': 24}
{'_id': '1994è1998', 'movie_count': 1}
{'_id': 1935, 'movie_count': 31}
{'_id': 1936, 'movie_count': 30}
{'_id': '1996è', 'movie_count': 1}
{'_id': 1913, 'movie_count': 1}
```

> Sort the above results in chronological order by adding a final `$sort` stage.

Update

You can update a document in a collection using the `update_one()` method. The first parameter of the `update_one()` method is a query object defining which document to update. The second parameter is an object defining the new values of the document.

Let's change the `title` of the document from `"Traffic in Souls"` to `"Traffic in Souls (1913)"`:

```
movies.update_one({'title': 'Traffic in Souls'}, {'$set': {'title': 'Traffic
```

```
UpdateResult({'n': 1, 'electionId': ObjectId('7fffffff000000000000008e'),
'opTime': {'ts': Timestamp(1779075796, 34), 't': 142}, 'nModified': 1, 'ok':
1.0, '$clusterTime': {'clusterTime': Timestamp(1779075796, 34), 'signature':
{'hash':
b'\xb4\xca#\x1b<\x8f\xc9\xbf\xc2\xe3\xa2\xc7\xc3\xa0\xb0r\xbd\x17\x90\xed',
'keyId': 7589030134825353218}}, 'operationTime': Timestamp(1779075796, 34),
'updatedExisting': True}, acknowledged=True)
```

```
movies.find_one({'_id': ObjectId('573a1390f29313caabcd4eaf')})
```

```
{'_id': ObjectId('573a1390f29313caabcd4eaf'),
 'plot': 'A woman, with the aid of her police officer sweetheart, endeavors
to uncover the prostitution ring that has kidnapped her sister, and the
philanthropist who secretly runs it.',
 'genres': ['Crime', 'Drama'],
 'runtime': 88,
 'cast': ['Jane Gail', 'Ethel Grandin', 'William H. Turner', 'Matt Moore'],
 'num_mflix_comments': 1,
 'poster': 'https://m.media-
```

```
amazon.com/images/M/MV5BYzk0YWQzMGYtYTM5MC00NjM2LWE5YzYtMjgyNDVhZDg1N2YzXkEyX
 'title': 'Traffic in Souls (1913)',
 'lastupdated': '2015-09-15 02:07:14.247000000',
 'languages': ['English'],
 'released': datetime.datetime(1913, 11, 24, 0, 0),
 'directors': ['George Loane Tucker'],
 'rated': 'TV-PG',
 'awards': {'wins': 1, 'nominations': 0, 'text': '1 win.'},
 'year': 1913,
 'imdb': {'rating': 6.0, 'votes': 371, 'id': 3471},
 'countries': ['USA'],
 'type': 'movie',
 'tomatoes': {'viewer': {'rating': 3.0, 'numReviews': 85, 'meter': 57},
  'dvd': datetime.datetime(2008, 8, 26, 0, 0),
  'lastUpdated': datetime.datetime(2015, 8, 10, 18, 33, 55)}}
```

> Update the same document's `lastUpdated` to the current date and time.

For updating multiple documents, you can use the `update_many` method.

Create

To insert a document into a collection, we can use the `insert_one()` method, and pass in a dictionary.

Likewise, we can use the `insert_many()` method to insert multiple documents into a collection.

```
movies.insert_one({'title': 'The Accountant', 'year': 2012})
```

```
InsertOneResult(ObjectId('6a0a8b0f952e5535866c6707'), acknowledged=True)
```

We could insert the document (dictionary) with "incomplete" fields as above. As a NoSQL database, MongoDB is considered schemaless because it does not require a rigid, pre-defined schema like a relational database.

Delete

To delete a document, we can use the `delete_one()` or `delete_many()` methods.

```
movies.delete_one({'title': 'The Accountant', 'year': 2012})
```

```
DeleteResult({'n': 1, 'electionId': ObjectId('7fffffff000000000000008e'),
'opTime': {'ts': Timestamp(1779075863, 22), 't': 142}, 'ok': 1.0,
'$clusterTime': {'clusterTime': Timestamp(1779075863, 22), 'signature':
```

```
{'hash': b'\xccK\xb8\xa4\x00YG\x0f\x05\xc9n\x9b\xbc\x1c\xea<\xdc\xb0/c',
'keyId': 7589030134825353218}}, 'operationTime': Timestamp(1779075863, 22)},
acknowledged=True)
```

Terminate cluster

To terminate your cluster, click the 3 dots next to your cluster name and click 'Terminate' - see this screenshot for example

Double-click (or enter) to edit

```
"""Basic connection example.
"""

import redis

r = redis.Redis(
    host='redis-11031.crce295.us-east-1-1.ec2.cloud.redislabs.com',
    port=11031,
    decode_responses=True,
    username="default",
    password="oDEA107V8qRvQAabjKU8XXxsUrKI1OKW",
)

success = r.set('foo', 'bar')
# True

result = r.get('foo')
print(result)
# >>> bar
```

```
bar
```